UNIVERSIDAD CENFOTEC



Programación Orientada a Objetos

PORTAFOLIO DE EVIDENCIAS

Primer avance

Romario Salas Cerdas

Silvia Calvo González

Fecha: 25 de octubre de 2026

Justificación

El sistema que es de interés para desarrollar en este portafolio es un sistema de proveeduría de un Colegio.

En este colegio lo que son los uniformes, cuadernos, utensilios como bolígrafos lápices y demás se vendían a las familias de los estudiantes que quisieran comprarlos. Por lo cual la proveeduría estaba abierta de lunes a viernes a un público variado que incluía: Personal Administrativo, profesores, estudiantes y padres de familia.

Desde un inicio el sistema empezó con un inventario manual en donde todos los viernes se hacía cierre de inventario y había que contar manualmente cuantos materiales quedaban, compararlo con las facturas que daba el sistema y corroborar que el inventario estaba correcto.

La problemática radica en que no siempre el inventario del sistema estaba “al día” así como a veces no siempre se podían facturar exactamente los materiales que se estaban vendiendo. O por ejemplo en el caso de los uniformes, cuando se hacía un cambio de talla era muy difícil rastrear esos cambios en el inventario.

Un conteo manual toma mucho tiempo y está sujeto a errores, se considera aquí el factor de cómo es posible hacerlo más efectivo para llevar el conteo del stock actual, saber si hay algún material que está bajo en reservas o si más bien no se ha movido. Y poder llevar un inventario diario en lugar de semana a semana ayuda a tener una imagen más clara del estado del departamento de proveeduría.

Un departamento como este se podría beneficiar de la POO ya que se encuentra formada de objetos con datos y acciones propias como lo son los productos, los clientes y las ventas. Cada producto cuenta con sus propios atributos y solo deberían sufrir modificaciones en casos controlados de entras y salidas de mercadería.

Conceptos utilizados en el portafolio y el código:

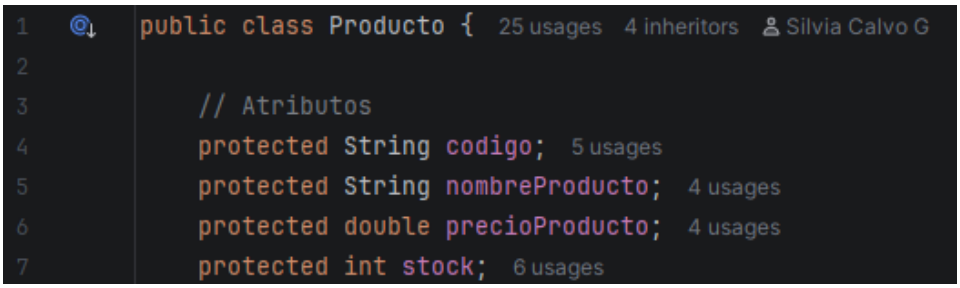
1. Abstracción

- Decisión: modelar únicamente los datos que importan a la proveeduría (código, nombre, precio y stock), sin detalles irrelevantes como el color de un lapicero.

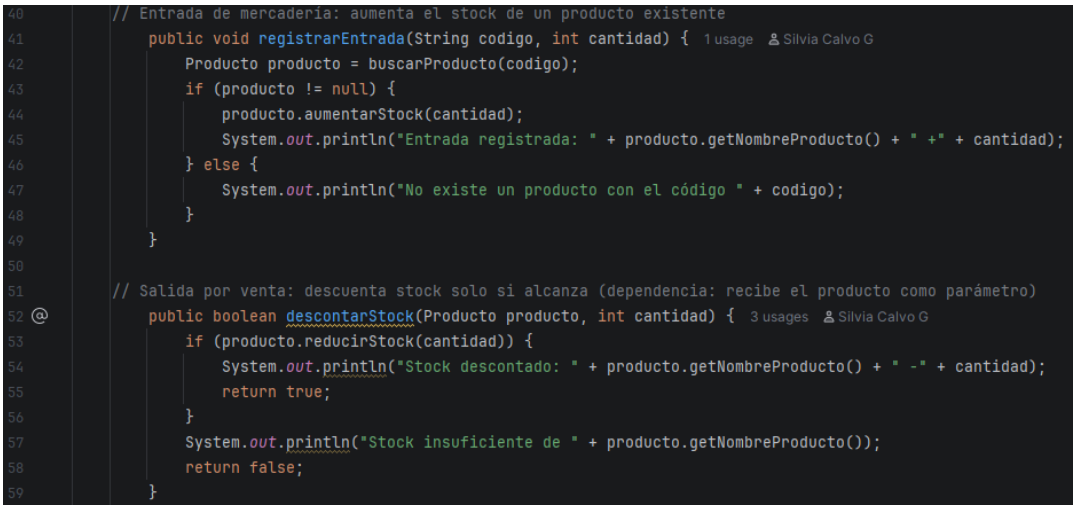
Razón: el sistema busca controlar el inventario y las ventas; todo lo demás es ruido.

2. Encapsulamiento

- Atributos private/protected: nadie fuera de la clase modifica los datos directamente; se usan métodos.



- Sin setStock: el stock solo cambia con aumentarStock y reducirStock. Esto responde al problema original del inventario desactualizado: si cualquiera pudiera escribir el stock, volvería el desorden del conteo manual.



- reducirStock devuelve boolean: impide vender más unidades de las que existen.
- Sin setters de listas: Canasta, Inventario y Venta no tienen setters para sus listas, para que nadie las reemplace completas ni las deje en null.
- Validaciones: Cliente.tipoValido y Uniforme.tipoValido aceptan solo valores permitidos.

```
14      // Getters
15      public ArrayList<Producto> getProductos() {  no usages  Silvia Calvo G
16          return productos;
17      }
18
19      // Setters - No tiene porque sino se podría reemplazar desde afuera y no se quiere eso
```

- Límite reconocido: los getters de listas devuelven la lista real, por lo que desde fuera se podría vaciar. Es una limitación honesta que se puede mencionar en la reflexión.

3. Atributos calculados en lugar de guardados

- Decisión: DetalleVenta.subtotal se calcula con precio × cantidad, y setCantidad lo recalcula.

  Razón: un dato derivado que se guarda a mano puede quedar desactualizado o ser incorrecto.

```
29      // Setters (recalcula subtotal para que no se quede desactualizado)
30  @   public void setProducto(Producto producto) {  no usages  Silvia Calvo G
31          this.producto = producto;
32          this.subtotal = producto.getPrecioProducto() * cantidad;
33      }
34
35      public void setCantidad(int cantidad) {  no usages  Silvia Calvo G
36          this.cantidad = cantidad;
37          this.subtotal = producto.getPrecioProducto() * cantidad;
38      }
```

4. Herencia

- Decisión: Producto → Uniforme / Libro → LibroLectura / LibroEjercicios.



Razón: todos los productos comparten código, nombre, precio y stock; las subclases solo definen lo nuevo.

- Beneficio: Inventario y Canasta guardan ArrayList<Producto> y aceptan cualquier tipo de producto.

```java
import java.util.ArrayList;

public class Inventario {  2 usages  Silvia Calvo G

    // Atributos
    private ArrayList <Producto> productos; // agregación: los productos existen por sí solos  6 usages
    private final int STOCK_MINIMO = 10; // constante: bajo este número se considera stock bajo  3 usages

    // Métodos
    // Constructor
    public Inventario () {  1 usage  Silvia Calvo G
        this.productos = new ArrayList<>();
    }
```

- super.toString(): cada subclase reutiliza el texto de su clase madre y agrega lo propio, sin repetir código.
- Uniforme con atributos: básico y deporte solo cambian un valor, no el comportamiento; crear dos clases sería excesivo.

Clases y sus relaciones

Qué hace cada una

Clase	Papel	Datos y acciones que ya mencionaste
Producto	Superclase	código, nombre, precio, stock
Uniforme	Hija de Producto	talla, tipo (básico/deporte)
Libro	Hija de Producto y madre de dos	idioma
LibroLectura	Hija de Libro	género
LibroEjercicios	Hija de Libro	materia
Canasta	Guarda lo que el cliente va eligiendo	lista de productos
Cliente	Quien compra	datos personales, comprar(Canasta)
Venta	Registro de una compra	fecha, cliente, detalles
DetalleVenta	Un renglón de la venta	producto, cantidad, subtotal
Inventario	Sabe cuántas unidades hay	lista de productos, stock bajo

Las tres relaciones, ya con tu decisión

Tipo	Par	Explicación
Agregación	Canasta ◇→ Producto	Los productos existen antes de entrar a la canasta
Composición	Venta ◆→ DetalleVenta	Sin la venta, sus renglones no existen
Dependencia	Cliente ⋯> Canasta	comprar(Canasta) la recibe como parámetro y no la guarda

Desafíos encontrados:

Desafío 1: Agregación vs. composición

Me costó diferenciar ambas relaciones porque en las dos una clase contiene objetos de otra. Lo resolví preguntándome si el objeto puede existir sin su contenedor. Un producto existe antes de entrar a la canasta, así que es agregación. Un DetalleVenta no tiene sentido sin su venta, y por eso Venta lo crea por dentro con new, lo que es composición. Aprendí que la diferencia está en quién crea el objeto y controla su ciclo de vida.

Desafío 2: Constructores con super( )

En las subclases de Libro, mis constructores llamaban a super con parámetros incompletos. Entendí que el constructor de la clase madre exige todos sus datos. Corregí las subclases para que reciban todo, pasen a super lo que corresponde y guarden solo sus atributos propios. Aprendí que una subclase no puede saltarse el constructor de su clase madre.

Desafío 3: Asignaciones que no hacían nada

En Cliente, el método setNombre asignaba el atributo a sí mismo, así que el valor nunca cambiaba y el programa no mostraba ningún error. Lo detecté probando los métodos desde el Main. Aprendí que un código que compila no necesariamente funciona bien, y que probar cada método con datos reales es parte del trabajo.

Desafío 4: Decidir qué no se puede modificar

En Producto no incluí setStock ni setCodigo. El stock solo cambia con aumentarStock y reducirStock, que validan la cantidad, y así evito datos incorrectos en el inventario. Aprendí que el encapsulamiento no es solo poner atributos privados, sino decidir qué operaciones tienen sentido para el negocio.

Material de complemento visual:

https://canva.link/dgc4c7o0ubc1i3k

Link del repositorio:

https://github.com/SilCG/ProveeduriaPortafolio.git